ETC3250/5250 Introduction to Machine Learning

Week 5: Decision Trees

Jack Jewson

Jack.Jewson@monash.edu

Department of Econometrics and Business Statistics

ETC3250/5250 Lecture 5

Overview

We will cover:

- Regularisation in linear Models
- Linear model feature engineering
- Classification trees, algorithm, stopping rules
- Decision tree variable importance

Regularisation

ETC3250/5250 Lecture 5

What is the problem in high-high-D? (1/3)

20 observations and 2 classes: A, B. One variable with separation, 19 noise variables

---

What will be the optimal Logistic Regression coefficients?

What is the problem in high-high-D? (2/3)

Fit logistic regression with the first two variables.

```
parsnip model object


Call:  stats::glm(formula = cl ~ ., family = stats::binomial, data =
data)

Coefficients:
(Intercept)           x1           x2
       3.12       -33.67         2.95

Degrees of Freedom: 19 Total (i.e. Null);  17 Residual
Null Deviance:      28
Residual Deviance: 5.5e-10  AIC: 6
```

Coefficient for `x1` MUCH higher than `x2`. As expected!

Predict the training and test sets

```
A B
```

```
A 10  0
B  0 10



  A B
A 5 0
B 0 5
```

Perfect!

ETC3250/5250 Lecture 5

What is the problem in high-high-D? (3/3)

What happens to the estimated coefficients as dimensions of noise increase?

Remember, the noise variables should have coefficient = ZERO.

How do you check? (1/2)

Permutation is your friend, for high-dimensional data analysis. Permute the class labels. and refit the model

```
set.seed(951)
tr_perm <- tr |>
  mutate(cl = sample(cl))
```

How do you check? (2/2)

- Permuting response, repeating the analysis, then make model summaries and diagnostic plots
- Comparing with test set is critical.
- If results (error/accuracy, low-d visual summary) on test set are very different than training, it could be due to high-dimensionality/overfitting.

How can you correct?

- Subset selection: reduce the number of variables before attempting to model
- Penalisation: change the optimisation criteria to include another term which makes it worse when there are more coefficients

Regularisation

Recall that in Logistic regression we set

$$\hat{\beta} := \arg\max_{\beta \in \mathbb{R}^{p+1}} \log l_n(\beta)$$

where $\beta = \{\beta_0, \beta_1\}$,

and $l_n(\beta_0, \beta)$ is the likelihood

$$l_n(\beta) := \prod_{i=1}^{n} P(Y_i = y_i | x, \beta)$$

Estimates the parameters of the model to make the data as likely as possible

Any change to a β_j that makes the data even a little more likely is preferable

Penalised logistic regressions sets

$$\hat{\beta} := \arg\max_{\beta \in \mathbb{R}^{p+1}} \log l_n(\beta) - \lambda H(\beta)$$

Where $H(\beta)$ is increasing away from 0 and so penalises increasing any β_j

ETC3250/5250 Lecture 5

If increasing a β_j only make the data a little more likely then this is no longer preferable

ETC3250/5250 Lecture 5

Regularisation

What happens to the estimated coefficients as dimensions of noise increase?

Remember, the noise variables should have coefficient = ZERO.

What penalty?

Typical penalties are

- $H(\beta) = \frac{1}{2}\sum_{j=1}^{p}\beta_j^2$ referred to as the ridge penalty
- $H(\beta) = \sum_{j=1}^{p}|\beta_j|$ referred to as the LASSO penalty - with added benefit of shrinking β_j's exactly to 0 (i.e. selecting variables)
- $H(\beta) = \alpha\sum_{j=1}^{p}|\beta_j| + \frac{(1-\alpha)}{2}\sum_{j=1}^{p}\beta_j^2$ referred to as the elastic net - tries to capture the best of both ridge and lasso
- Elastic-net with $\alpha = 1$ is LASSO and $\alpha = 0$ is ridge

---

In `tidymodels`

https://parsnip.tidymodels.org/reference/logistic_reg.html

```
logistic_mod <- logistic_reg(penalty = lambda, mixture = alpha) |
set_engine("glmnet") |>
set_mode("classification")
```

$\lambda = 0$ and $\alpha = 0$

```
 [1] -0.041 -1.661  0.179 -0.119  0.020  0.299  0.058 -0.440
 [9]  0.312 -0.464  0.293 -0.123  0.061 -0.195  0.174  0.133
[17] -0.434 -0.169  0.366  0.089 -0.056
```

$\lambda = 0.5$ and $\alpha = 0$

```
 [1] -0.0055 -0.5613  0.0613 -0.0488 -0.0131  0.1156 -0.0050
 [8] -0.2163  0.1059 -0.2118  0.1377 -0.0350  0.0610 -0.0813
[15]  0.0462  0.0609 -0.1993 -0.0551  0.1048  0.0608 -0.0056
```

$\lambda = 0.1$ and $\alpha = 1$

```
 [1]  0.0062 -2.2600  0.0000  0.0000  0.0000  0.0000  0.0000
 [8]  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000
[15]  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000
```

$\lambda = 0.001$ and $\alpha = 1$

```
 [1]  0.37 -7.84  0.00  0.00  0.00  0.00  0.00  0.00  0.00
[10] -0.12  0.00  0.00  0.00  0.00  0.00  0.00  0.00  0.00
[19]  0.00  0.00  0.00
```

$\lambda = 0.1$ and $\alpha = 0.5$

```
 [1]  0.0087 -1.7182  0.0000  0.0000  0.0000  0.0000  0.0000
 [8] -0.1695  0.0000 -0.1532  0.0000  0.0000  0.0000  0.0000
[15]  0.0000  0.0000 -0.1048  0.0000  0.0000  0.0000  0.0000
```

ETC3250/5250 Lecture 5

Standardisation

- Unlike in kNN, we said that standardisation wasn't necessary for logistic regression
- But that it did help interpretation,
- as it made sure all of the coefficients β_j were on the same scale
- We are now regularising our logistic regression models by penalising the size of their coefficients
- and we only have two hyperparameter λ and α to do this
- λ and α are the same for each parameter β_j
- So beneficial to have all β_j's on the same scale
- For penalised logistic regression you SHOULD standardise your data

The regularisation hyperparameters

How do you think we set λ (and α)?

Set λ and α by cross-validation - use the regularisation that gives the optimal out of sample performance

First we standardise the data

```
tr <- tr |>
  mutate_if(is.numeric, function(x) (x-mean(x))/sd(x))
```

```
tune_spec <-
  logistic_reg(
    penalty = tune(),
    mixture = tune()
  ) |>
  set_mode("classification") |>
  set_engine("glmnet")
```

ETC3250/5250 Lecture 5

```r
hyperparam_grid <- grid_regular(penalty(), mixture(),
                                levels = 5)
```

```r
set.seed(234)
p_folds <- vfold_cv(tr, v = 5, strata = cl)
```

```r
log_reg_wf <- workflow() |>
  add_model(tune_spec) |>
  add_formula(cl~.)
```

ETC3250/5250 Lecture 5

The regularisation hyperparameters

```
set.seed(345)
log_reg_res <-
  log_reg_wf |>
  tune_grid(
    resamples = p_folds,
    grid = hyperparam_grid,
    metrics = NULL #NULL to compute a standard set of metrics.
    )
```

```
log_reg_res |>
  select_best(metric = "roc_auc")
```

```
# A tibble: 1 × 3
  penalty mixture .config
```

ETC3250/5250 Lecture 5

```
    <dbl>   <dbl> <chr>
1       1    0.25 Preprocessor1_Model10
```

Refitting the logistic regression model with the best hyperparameters to the whole training data

```
tidy(w_log_reg)  |> pull(estimate)
```

```
 [1] -8.0e-06 -2.5e-01  0.0e+00  0.0e+00  0.0e+00  0.0e+00
 [7]  0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00
[13]  0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00  0.0e+00
[19]  0.0e+00  0.0e+00  0.0e+00
```

ETC3250/5250 Lecture 5

Feature engineering

ETC3250/5250 Lecture 5

Feature engineering

Some times to get a better fit you need a more complex model
Other times you can get the same results by transforming the original variables (See `tidymodels` steps.)
Can we transform for the variables to satisfy the linear assumptions

- scaling, centering, sphering (`step_pca`)
- log or square root or box-cox transformation (`step_log`)
- ratio of values (`step_ratio`)
- **polynomials or splines: x_1^2, x_1^3 (`step_ns`)**
- dummy variables/one-hot-encoding: categorical predictors expanded into multiple new binary variables (`step_dummy`)

Example

Example

Trees

Nice explanation of trees, forests, boosted trees

What is a Decision Tree

We though about logistic regression as dividing the predictor space up into K regions, one for each response
Decision trees use a tree structure to divide up the feature space into regions with similar response behaviour
And predict the same response value (or probability) for observation in the same part of the tree

Consider trying to predict whether a customer would default on their credit card loan from their account balance and income

```
# A tibble: 5 × 4
  ID    Default Income Balance
  <chr> <chr>    <dbl>   <dbl>
1 obs1  No          71      26
2 obs2  No          65      30
3 obs3  Yes         55      36
```

ETC3250/5250 Lecture 5

```
4 obs4  No          76      45
5 obs5  Yes         68      53
```

ETC3250/5250 Lecture 5

What is a Decision Tree

```
# A tibble: 5 × 4
  ID    Default Income Balance
  <chr> <chr>    <dbl>   <dbl>
1 obs1  No          71      26
2 obs2  No          65      30
3 obs3  Yes         55      36
4 obs4  No          76      45
5 obs5  Yes         68      53
```

Each **node** of a decision tree takes one variable and splits it at one point

---

ETC3250/5250 Lecture 5

What is a Decision Tree

```
# A tibble: 5 × 4
  ID    Default Income Balance
  <chr> <chr>    <dbl>   <dbl>
1 obs1  No          71      26
2 obs2  No          65      30
3 obs3  Yes         55      36
4 obs4  No          76      45
5 obs5  Yes         68      53
```

What is a Decision Tree

```
# A tibble: 5 × 4
  ID    Default Income Balance
  <chr> <chr>    <dbl>   <dbl>
1 obs1  No          71      26
2 obs2  No          65      30
3 obs3  Yes         55      36
4 obs4  No          76      45
5 obs5  Yes         68      53
```

ETC3250/5250 Lecture 5

What is a Decision Tree

ETC3250/5250 Lecture 5

What is a Decision Tree

When a new observation arrives e.g. with `Income = 67` and `Balance = 20` we pass this observation through the tree.
`Income = 67` so Income > 64 is **TRUE**
`Balance = 20` so Balance < 50 is also **TRUE**
Then use the leaf node it appears in to predict its response and predict **NOT DEFAULT**

What is a Decision Tree

Decision trees use a tree structure to divide up the feature space into regions with similar response behaviour

What is a Decision Tree

Decision trees use a tree structure to divide up the feature space into regions with similar response behaviour

What is a Decision Tree

Decision trees use a tree structure to divide up the feature space into regions with similar response behaviour

What is a Decision Tree

Mathematically we can write a decision tree machine learning model as

$$g(x;\beta,R) = \sum_{m=1}^{M} \beta_m \mathbb{I}(x \in R_m)$$

where $\mathbb{I}(x \in R_m) = 1$ if $x \in R_m$ and $\mathbb{I}(x \notin R_m) = 0$ otherwise.
For (binary) classification $P(Y=1;x) = \frac{1}{1+e^{-g(x;\beta,R)}}$ (the logistic function)
For regression $\mathbb{E}[Y;x] = g(x;\beta,R)$

It is interesting to contrast this with (generalsied) linear models which sets $P(Y=1;x) = \frac{1}{1+e^{-x^\top\beta}}$ for classification and $\mathbb{E}[Y;x] = x^\top\beta$ for regression.
The non-linear (non-parametric) tree function $g(x;\beta,R)$ takes the place of the linear predictor $x^\top\beta$

ETC3250/5250 Lecture 5

What is a Decision Tree

In R_1, the probability of defaulting is $P(Y=1|x) = \frac{1}{1+e^{-10}} = 0.9999$
In R_2, the probability of defaulting is $P(Y=1|x) = \frac{1}{1+e^{9.5}} = 0.0000748$
In R_3, the probability of defaulting is $P(Y=1|x) = \frac{1}{1+e^{9}} = 0.99988$

ETC3250/5250 Lecture 5

Fitting a Decision Tree

Fitting a decision tree to data requires

- Determining the regions $R_1, R_2, \ldots$ i.e. which variables to split and where
- Estimating the parameters for each region i.e. $\beta_1, \beta_2, \ldots$

Estimating the $\beta_1, \beta_2, \ldots$ for fixed values of the $R_1, R_2, \ldots$ is quite straightforward. We can use Maximum likelihood restricted for the training observations within each leaf node R_m.
Determining the regions $R_1, R_2, \ldots$ is more challenging. We will consider several different algorithms to do this
The idea, is to find the variable to split and the point to split by that makes the observations on the same branches after that split as similar as possible

CART Algorithm: growing a tree

CART: Classification And Regression Trees

1. All observations in a single set
2. Sort values on first variable
3. Compute the chosen split criteria for all possible splits into two sets
4. Choose the best split on this variable. Save this info.
5. Repeat 2-4 for all other variables
6. Choose the best variable to split on, based on the best split. Your data is now in two sets.
7. Repeat 1-6 on each subset.
8. Stop when stopping rule that decides that the best classification model is achieved.

Pros and cons:

- Trees are a very flexible way to fit a classifier.

- They can
  - utilise different types of predictor variables
  - ignore missing values
  - handle different units or scales on variables
  - capture intricate patterns
- However, they operate on a per variable basis, and do not effectively model separation when a combination of variables is needed.
- Can be too flexible and need to be regularised to avoid overfitting

Common split criteria

We want to split in ways the encourage observation with similar responses into the same split i.e. the child nodes have minimal variability in y

Classification

- The Gini index measures is defined as:
  $$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$
  where $\hat{p}_{mk}$ is the proportion of observation in subset m of the data that belong to class k
- Gini is maximised at $\hat{p}_{mk} = 0.5$ i.e. a 50:50 split, and minimised when $\hat{p}_{mk} = 0$ or 1
- (Note $p(1-p)$ is the variance of a Bernoulli distribution with success probability p)

Split the data where combining Gini for left bucket (Gini_L) and right bucket (Gini_R), makes the biggest reduction from the overall Gini.

ETC3250/5250 Lecture 5

Common split criteria

We want to split in ways the encourage observation with similar responses into the same split i.e. the child nodes have minimal variability in y

Regression

Define

$$\mathrm{Var}_m = \frac{1}{n_m} \sum_{i=1}^{n_m} (y_{mi} - \bar{y}_m)^2$$

where $\bar{y}_m$ is the average response in subset m

Split the data where combining Variance for left bucket (Var_L) and right bucket (Var_R), makes the biggest reduction from the overall Variance.

Illustration (1/2)

x	cl
11	A
33	A
39	B
44	A
50	A
56	B
70	B

Note: x is sorted from lowest to highest!
All possible splits shown by vertical lines

What do you think is the best split? 1, 2, 3, 4, 5, or 6??

ETC3250/5250 Lecture 5

You can see here that scale doesn’t matter

ETC3250/5250 Lecture 5

Illustration (2/2)

Calculate the impurity for split 5

The left bucket is

x	cl
11	A
33	A
39	B
44	A
50	A

and the right bucket is

x	cl
56	B
70	B

Using Gini $G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$

Left bucket:

$\hat{p}_{LA} = 4/5 = 0.8, \;\; \hat{p}_{LB} = 1/5 = 0.2$

$G_L = 0.8(1 - 0.8) + 0.2(1 - 0.2) = 0.32$

Right bucket:

$\hat{p}_{RA} = 0/2 = 0, \;\; \hat{p}_{RB} = 2/2 = 1$

$G_R = 0(1 - 0) + 1(1 - 1) = 0$

Combine with weighted sum to get impurity for the split:

$p_L G_L + p_R G_R = 0.23$

where $p_L = 5/7$ and $p_R = 2/7$

Your turn: Compute the impurity for split 2.

ETC3250/5250 Lecture 5

Categorical Predictors

Splits on categorical variables

We can construct a split of one category vs the other e.g. {`koala`} vs {`roo`, `emu`}

Possible best split would be if koala then assign to Vic else assign to WA, because Vic has more koalas but and WA has more emus and roos.

Dealing with missing values on predictors

x1	x2	x3	x4	y
19	-8	22	-24	A
NA	-10	26	-26	A
15	NA	32	-27	B
17	-6	27	-25	A
18	-5	NA	-23	A
13	-3	37	NA	B
12	-1	35	-30	B
11	-7	24	-31	B

50% of cases have missing values.

Default method to deal with missing data is either

- Remove any variable (column) with a high proportion of missing data

- Remove any observation (row) with a missing value i.e. obs2, obs3, obs5 and obs7 (i.e. half the data)
- Or *impute* missing observations using their columns mean i.e. obs2 $\mapsto (\bar{x}_1, -10, 26, -26, A)$

ETC3250/5250 Lecture 5

Dealing with missing values on predictors

x1	x2	x3	x4	y
19	-8	22	-24	A
NA	-10	26	-26	A
15	NA	32	-27	B
17	-6	27	-25	A
18	-5	NA	-23	A
13	-3	37	NA	B
12	-1	35	-30	B
11	-7	24	-31	B

50% of cases have missing values.

Trees only work with a single variable at a time, so can easily just ignore a variable when calculating the plit criteria.

When predicting an observation with missing values, imputation is required

ETC3250/5250 Lecture 5

Example: penguins 1/6

See **https://parsnip.tidymodels.org/reference/decision_tree.html**

```
set.seed(1156)
p_split <- initial_split(p_sub, 2/3, strata=species)
p_tr <- training(p_split)
p_ts <- testing(p_split)
```

```
tree_spec <- decision_tree() |>
  set_mode("classification") |>
  set_engine("rpart")
```

```
p_fit_tree <- tree_spec |>
  fit(species~., data=p_tr)
```

```
p_fit_tree
```

```
parsnip model object

n= 145

node), split, n, loss, yval, (yprob)
      * denotes terminal node
```

```
1) root 145 45 Adelie (0.690 0.310)
  2) bl< 43 99  2 Adelie (0.980 0.020) *
  3) bl>=43 46  3 Chinstrap (0.065 0.935) *
```

ETC3250/5250 Lecture 5

Example: penguins 2/6

ETC3250/5250 Lecture 5

Example: penguins 3/6

ETC3250/5250 Lecture 5

Stopping rules

- Minimum split: number of observations in a node, in order for a split to be made
- Tree depth: The maximum number of splits between the a terminal node and root node
- Complexity parameter: minimum difference between impurity values required to continue splitting

These are all forms of regularisation!
They aim to reduce the in-sample fit in the hope it improves out-of-sample fit i.e. prevents overfitting.

Example: penguins 4/6

Defaults for `rpart` are:

```
rpart.control(minsplit = 20,
  cp = 0.01,
  maxdepth = 30,
  ...)
```

```
tree_spec <- decision_tree(min_n = 10) |>
  set_mode("classification") |>
  set_engine("rpart")

p_fit_tree <- tree_spec |>
  fit(species~., data=p_tr)
```

```
p_fit_tree
```

```
parsnip model object

n= 145
```

```
node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 145 45 Adelie (0.690 0.310)
   2) bl< 43 99  2 Adelie (0.980 0.020)
     4) bl< 41 75  0 Adelie (1.000 0.000) *
     5) bl>=41 24  2 Adelie (0.917 0.083)
      10) bm>=3.4e+03 21  0 Adelie (1.000 0.000) *
      11) bm< 3.4e+03 3  1 Chinstrap (0.333 0.667) *
   3) bl>=43 46  3 Chinstrap (0.065 0.935)
     6) bl< 46 10  3 Chinstrap (0.300 0.700)
```

ETC3250/5250 Lecture 5

Example: penguins 5/6

```
p_fit_tree |>
  extract_fit_engine() |>
  rpart.plot(type=3, extra=1)
```

Example: penguins 6/6

Model fit summary

```
# A tibble: 1 × 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary         0.946


# A tibble: 2 × 4
# Groups:   species [2]
  species   Adelie Chinstrap Accuracy
  <fct>      <int>     <int>    <dbl>
1 Adelie        50         1    0.980
2 Chinstrap      3        20    0.870


# A tibble: 1 × 3
  .metric      .estimator .estimate
  <chr>        <chr>          <dbl>
1 bal_accuracy binary         0.925
```

Model-in-the-data-space

ETC3250/5250 Lecture 5

ETC3250/5250 Lecture 5

Comparison with logistic regression

Tree model

Data-driven, only split on single variables

Logistic regression model

oblique splits (combination of variables)

Comparison with logistic regression

Tree model

Logistic regression model

Learning tree hyperparameters

For given stopping rules, the CART algorithm learns the splits and the reponse parameter in each terminal node.
However, it doesn't tell us what stopping rule to use, i.e. which values of `minsplit`, `cp`, and `maxdepth` to use
These are our tree hyper parameters that we learn by cross-validation
Cross-validation splits the **training data** into disjoint subset, and iteratively uses one as a *mini* test set the evaluate the model fitted on the remaining sets

Cross-validation in the `tidymodels`

```r
tune_spec <-
  decision_tree(
    cost_complexity = tune(),
    tree_depth = tune(),
    min_n = tune()
  ) |>
  set_engine("rpart") |>
  set_mode("classification")
```

```r
tree_grid <- grid_regular(cost_complexity(),
                          tree_depth(),
                          min_n(),
                          levels = 5)
```

```r
unique(tree_grid$cost_complexity)
```

```
[1] 1.0e-10 1.8e-08 3.2e-06 5.6e-04 1.0e-01
```

ETC3250/5250 Lecture 5

```r
unique(tree_grid$tree_depth)
```

```
[1]  1  4  8 11 15
```

```r
unique(tree_grid$min_n)
```

```
[1]  2 11 21 30 40
```

```r
set.seed(234)
p_folds <- vfold_cv(p_tr, v = 5, strata = species)
```

```r
set.seed(345)

tree_wf <- workflow() |>
  add_model(tune_spec) |>
  add_formula(species~.)
```

```r
tree_res <-
  tree_wf |>
  tune_grid(
    resamples = p_folds,
```

ETC3250/5250 Lecture 5

```
    grid = tree_grid,
    metrics = NULL #NULL to compute a standard set of metrics.
    )
```

```
tree_res |>
  collect_metrics() |>
  slice_head(n = 6)
```

```
# A tibble: 6 × 9
  cost_complexity tree_depth min_n .metric .estimator   mean
            <dbl>      <int> <int> <chr>   <chr>       <dbl>
1    0.0000000001          1     2 accura… binary     0.959
2    0.0000000001          1     2 brier_… binary     0.0405
3    0.0000000001          1     2 roc_auc binary     0.952
4    0.0000000178          1     2 accura… binary     0.959
5    0.0000000178          1     2 brier_… binary     0.0405
6    0.0000000178          1     2 roc_auc binary     0.952
# ℹ 3 more variables: n <int>, std_err <dbl>, .config <chr>
```

ETC3250/5250 Lecture 5

Cross-validation in the `tidymodels`

```
tree_res |>
  show_best(metric = "roc_auc")
```

```
# A tibble: 5 × 9
  cost_complexity tree_depth min_n .metric .estimator  mean
            <dbl>      <int> <int> <chr>   <chr>      <dbl>
1    0.0000000001          1     2 roc_auc binary     0.952
2    0.0000000178          1     2 roc_auc binary     0.952
3    0.00000316            1     2 roc_auc binary     0.952
4    0.000562              1     2 roc_auc binary     0.952
5    0.1                   1     2 roc_auc binary     0.952
# ℹ 3 more variables: n <int>, std_err <dbl>, .config <chr>
```

```
tree_res |>
  select_best(metric = "roc_auc")
```

```
# A tibble: 1 × 4
  cost_complexity tree_depth min_n .config
```

ETC3250/5250 Lecture 5

```
              <dbl>      <int> <int> <chr>
1    0.0000000001          1     2 Preprocessor1_Model001
```

For more information see - https://www.tidymodels.org/start/tuning/

ETC3250/5250 Lecture 5

Argument names

rpart

- Minimum split = `minsplit`
- Tree depth = `maxdepth`
- Complexity parameter = `cp`

```
tree_spec <- decision_tree() |>
  set_mode("classification") |>
  set_engine("rpart",
             control = rpart.control(minsplit = 10),
             model=TRUE)
```

tidymodels

- Minimum split = `min_n`
- Tree depth = `tree_depth`
- Complexity parameter = `cost_complexity`

ETC3250/5250 Lecture 5

```r
#| eval: false

tree_spec <-
  decision_tree(
    min_n = 10
  ) |>
  set_engine("rpart") |>
  set_mode("classification")
```

ETC3250/5250 Lecture 5

Variable importance (1/2)

Decision trees automatically do variable selection
For each split, every variable (and every split) is tested to see which one produces the best split
Uninformative variables will just not be picked

Each split only depends on one variable
So we can track the importance of each variable by tracking how much splits based on that variable decreased the split criteria

Gini importance adds up the difference in impurity value of the descendant nodes with the parent node for any split involving that variable. Quick to compute.

```
p_fit_tree$fit$variable.importance
```

```
  bl   bm
55.6  6.6
```

ETC3250/5250 Lecture 5

ETC3250/5250 Lecture 5

Decision tree limitations

Decision trees provide us with an extremely powerful machine learning model
It can capture complex non-linear dynamics, using a tree to divide up the predictor space and group similar training observations togther.
A major limitation of decision trees is that they are too flexible - it is very easy for them to perfectly classify the training data
This means they can very easily overfit the data - learn about the noise in the observed data rather than the signal in the process that generated the data
Learning the stopping criteria via cross-validation prevents this overfitting, but often leads to conservative models that don’t fully exploit the DTs power

Next: Forests, Boosting, Bias and Variance

ETC3250/5250 Lecture 5